

NAME : Shri Harshini Devi . A

REG.NO : 192524104

Dept. Name : B.Tech-AIDS

Course Code/Course Name: DSA0602 /Data Handling and Visualization

Date: 12/08/2026

Unit IV Activity II

1. A retail chain's finance team wants to understand how daily sales have moved over the past

two years before setting next year's budget. Using R, plot the daily sales as a time series,

label the axes, and identify any obvious long-term upward or downward movement.

Create dates for two years

```
Date <- seq(as.Date("2024-01-01"), as.Date("2025-12-31"), by = "day")
```

Create sample daily sales

```
set.seed(123)
```

```
Daily_Sales <- 5000 + (1:length(Date)) * 5 + rnorm(length(Date), 0, 500)
```

Create data frame

```
sales <- data.frame(Date, Daily_Sales)
```

Plot daily sales

```
plot(sales$Date, sales$Daily_Sales,
```

```
type = "l",
```

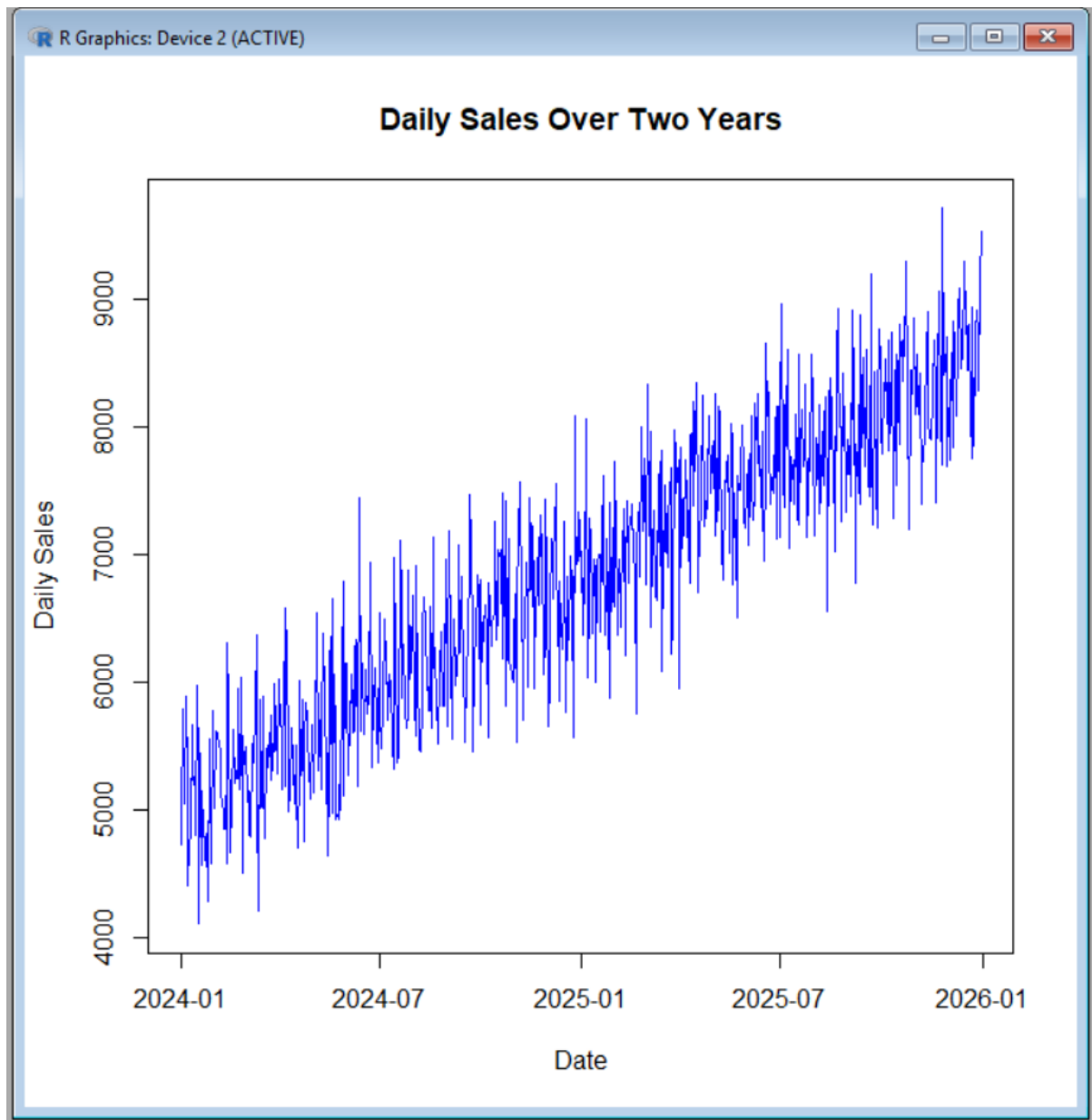
```
main = "Daily Sales Over Two Years",
```

```
xlab = "Date",
```

```
ylab = "Daily Sales",  
col = "blue")
```

```
# Add trend line
```

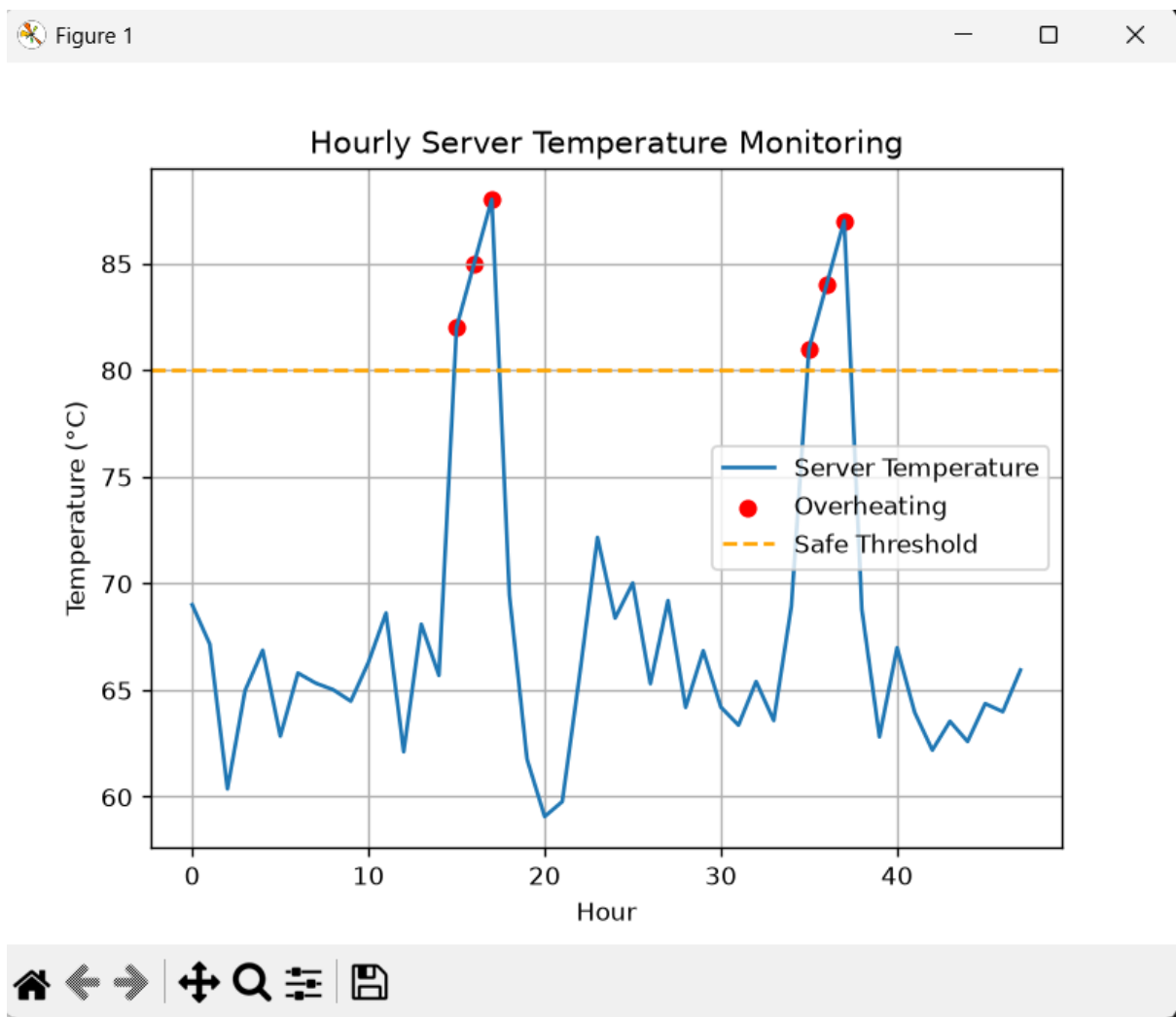
```
abline(lm(Daily_Sales ~ Date, data = sales),  
col = "red", lwd = 2)
```



2. An IoT company monitors hourly server temperature at a data center to catch overheating

risks early. Using Python, plot the temperature readings as a time series and highlight any

periods where values exceed a safe threshold.



3. A hedge fund analyst is comparing the daily closing prices of three competing tech stocks

to decide which one shows steadier growth. Using R, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

Create dates for 30 days

```
Date <- seq(as.Date("2025-01-01"), by = "day", length.out = 30)
```

Create sample closing prices

```
set.seed(123)
```

```
Stock_A <- 100 + cumsum(rnorm(30, 1, 2))
```

```
Stock_B <- 120 + cumsum(rnorm(30, 1, 1))
```

```
Stock_C <- 90 + cumsum(rnorm(30, 1, 4))
```

```
# Plot all three stocks
```

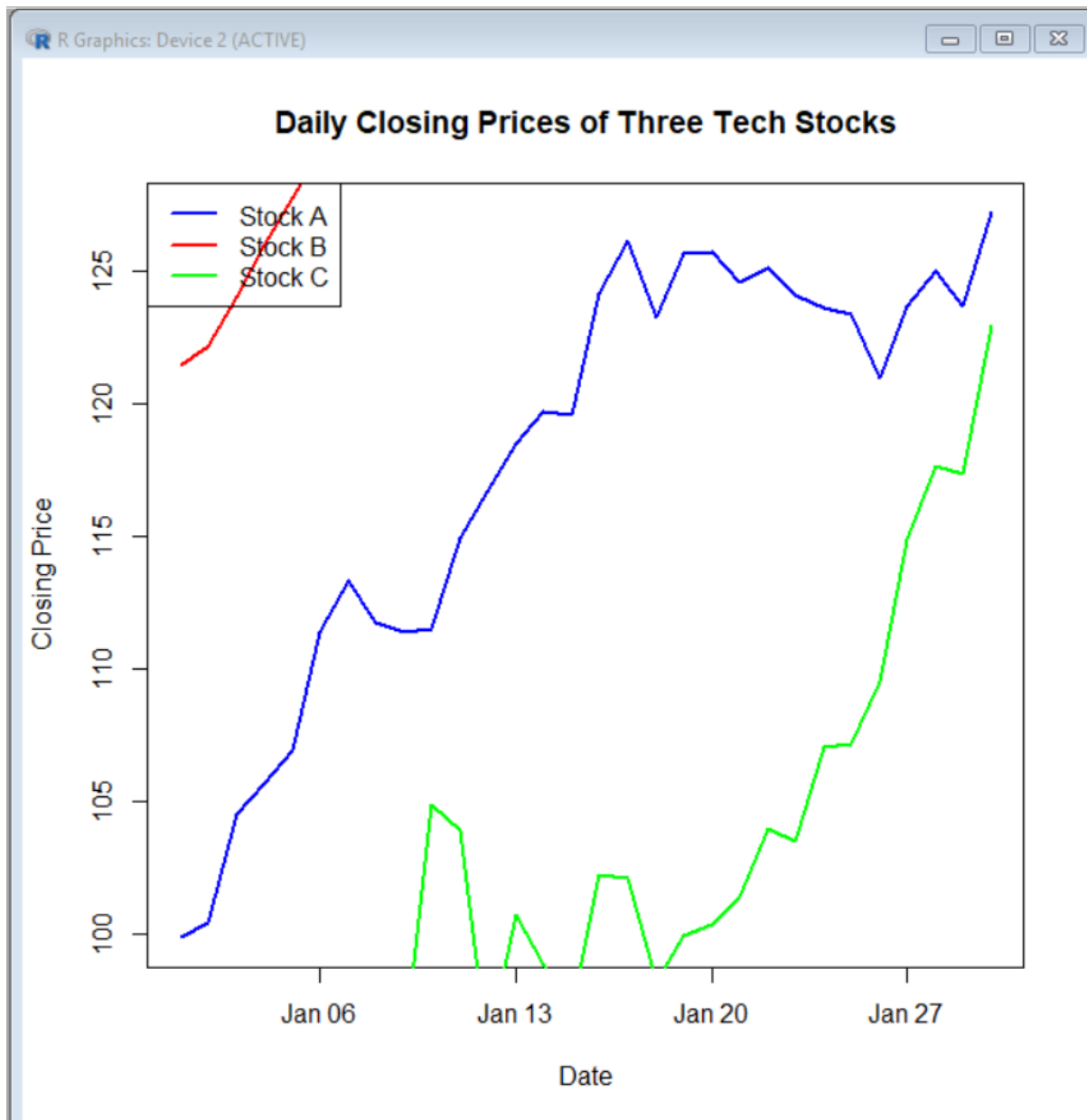
```
plot(Date, Stock_A,  
     type = "l",  
     col = "blue",  
     lwd = 2,  
     main = "Daily Closing Prices of Three Tech Stocks",  
     xlab = "Date",  
     ylab = "Closing Price")
```

```
lines(Date, Stock_B, col = "red", lwd = 2)
```

```
lines(Date, Stock_C, col = "green", lwd = 2)
```

```
# Add legend
```

```
legend("topleft",  
      legend = c("Stock A", "Stock B", "Stock C"),  
      col = c("blue", "red", "green"),  
      lwd = 2)
```



4. A city's energy board wants to compare monthly electricity consumption across four

different neighborhoods to plan targeted conservation campaigns. Using Python, plot the four

time series together and identify the neighborhood with the most erratic consumption pattern.

```
import numpy as np

import matplotlib.pyplot as plt

# Months

months = np.arange(1, 13)

# Sample monthly electricity consumption (kWh)

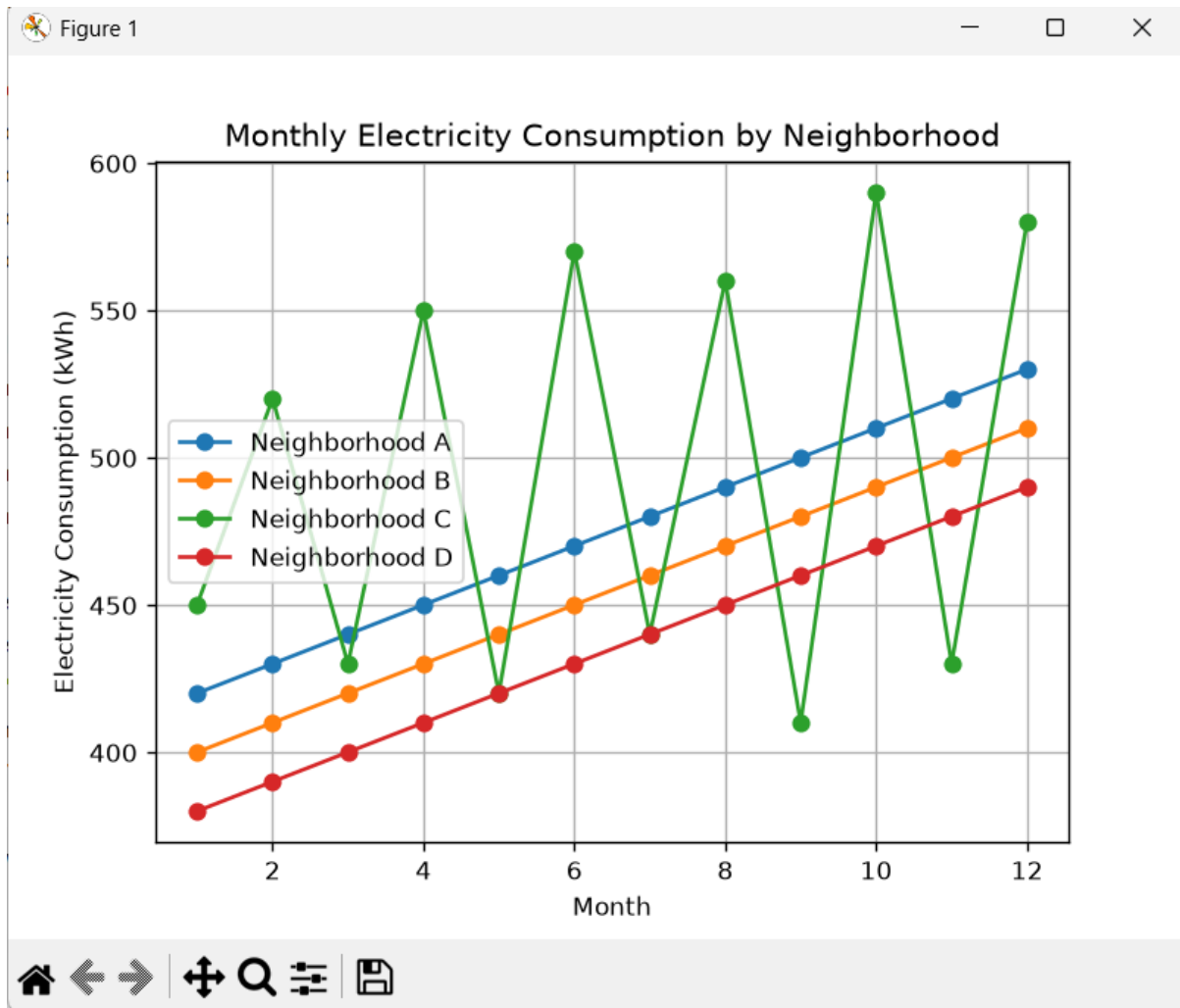
Neighborhood_A = [420, 430, 440, 450, 460, 470, 480, 490, 500, 510, 520, 530]
Neighborhood_B = [400, 410, 420, 430, 440, 450, 460, 470, 480, 490, 500, 510]
Neighborhood_C = [450, 520, 430, 550, 420, 570, 440, 560, 410, 590, 430, 580]
Neighborhood_D = [380, 390, 400, 410, 420, 430, 440, 450, 460, 470, 480, 490]

# Plot all four time series

plt.plot(months, Neighborhood_A, marker='o', label='Neighborhood A')
plt.plot(months, Neighborhood_B, marker='o', label='Neighborhood B')
plt.plot(months, Neighborhood_C, marker='o', label='Neighborhood C')
plt.plot(months, Neighborhood_D, marker='o', label='Neighborhood D')

plt.xlabel("Month")
plt.ylabel("Electricity Consumption (kWh)")
plt.title("Monthly Electricity Consumption by Neighborhood")
plt.legend()
plt.grid(True)

plt.show()
```



5. A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using R, plot a dose–response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

```
# Sample dose and patient response data
```

```
dose <- c(1, 2, 5, 10, 20, 30, 40, 50, 60, 80, 100)
```

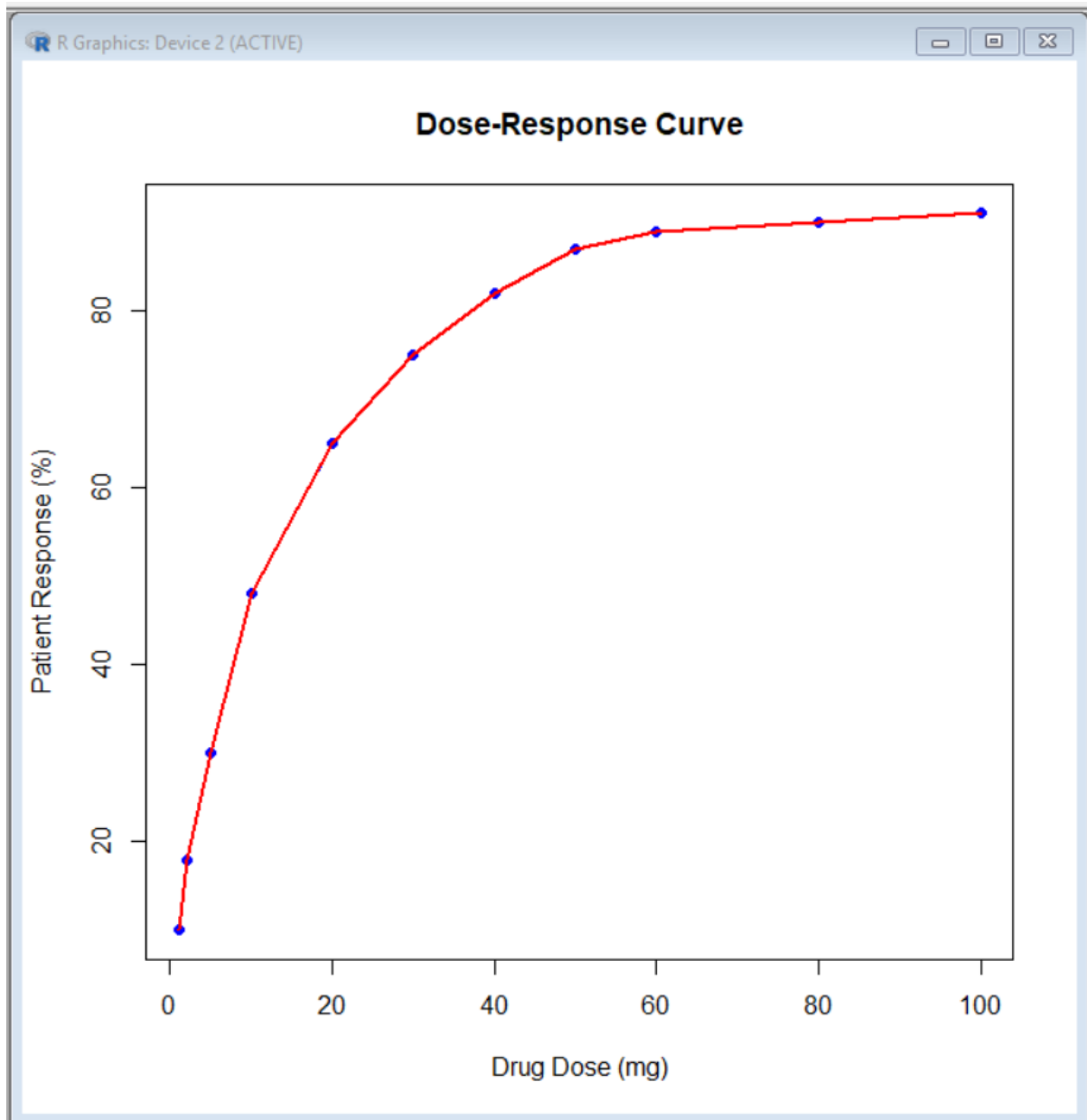
```
response <- c(10, 18, 30, 48, 65, 75, 82, 87, 89, 90, 91)
```

```
# Plot dose-response curve
```

```
plot(dose, response,  
     type = "b",  
     pch = 19,  
     col = "blue",  
     main = "Dose-Response Curve",  
     xlab = "Drug Dose (mg)",  
     ylab = "Patient Response (%)")
```

```
# Add a smooth curve
```

```
lines(dose, response,  
      col = "red",  
      lwd = 2)
```

6. An agricultural research lab is studying how increasing fertilizer concentration affects crop

yield, aiming to avoid wasteful over-application. Using Python, fit and plot a dose-response

curve of fertilizer amount vs. yield, and advise the optimal dosage range.

```
import numpy as np

import matplotlib.pyplot as plt

# Fertilizer amount (kg/hectare)
fertilizer = np.array([0, 20, 40, 60, 80, 100, 120, 140, 160, 180, 200])

# Sample crop yield (tons/hectare)
yield_data = np.array([2.0, 2.8, 3.7, 4.5, 5.2, 5.7,
                        6.0, 6.1, 6.15, 6.17, 6.18])

# Fit a polynomial curve
coefficients = np.polyfit(fertilizer, yield_data, 3)
model = np.poly1d(coefficients)

# Smooth values for the fitted curve
x = np.linspace(0, 200, 200)
y = model(x)

# Plot original data
plt.scatter(fertilizer, yield_data,
            color="blue", label="Observed Yield")

# Plot fitted curve
plt.plot(x, y,
         color="red", label="Fitted Dose-Response Curve")

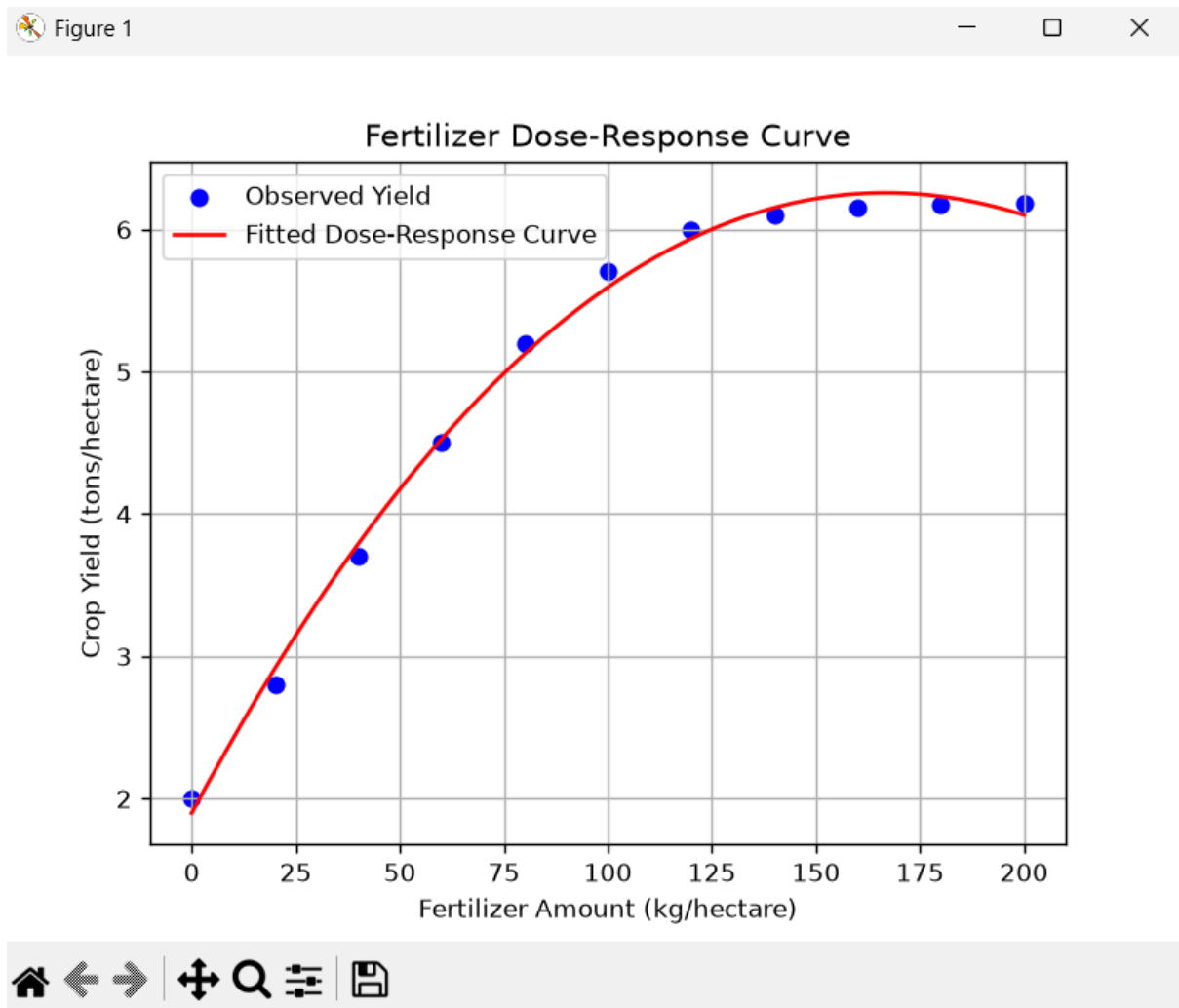
plt.xlabel("Fertilizer Amount (kg/hectare)")
plt.ylabel("Crop Yield (tons/hectare)")
```

```
plt.title("Fertilizer Dose-Response Curve")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



7. An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using R, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

```
# Create monthly dates for 3 years
Date <- seq(as.Date("2023-01-01"),
            as.Date("2025-12-01"),
            by = "month")

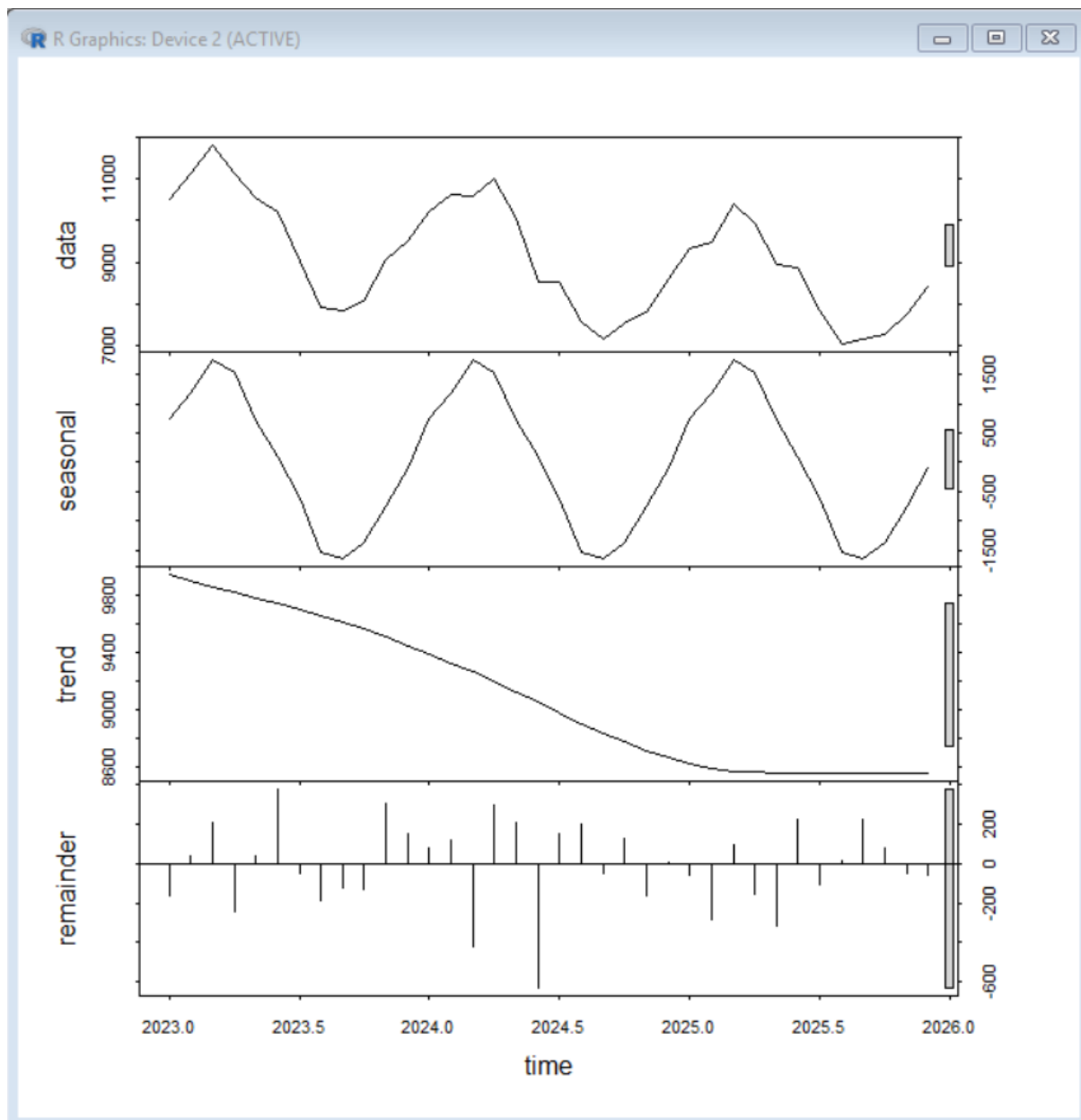
# Create sample ticket sales with seasonality and slow decline
set.seed(123)

Sales <- 10000 -
  (1:length(Date)) * 50 +
  1500 * sin(2 * pi * (1:length(Date)) / 12) +
  rnorm(length(Date), 0, 300)

# Create time series object
sales_ts <- ts(Sales,
               start = c(2023, 1),
               frequency = 12)

# Apply STL decomposition
stl_result <- stl(sales_ts, s.window = "periodic")

# Plot STL decomposition
plot(stl_result)
```



8. A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using Python, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import STL

# Create 2 years of weekly dates
dates = pd.date_range(start="2024-01-07",
                      periods=104,
                      freq="W")

# Create sample weekly active users
np.random.seed(123)

weeks = np.arange(104)

# Slow downward trend + seasonal pattern + random noise
users = (50000
        - weeks * 50
        + 3000 * np.sin(2 * np.pi * weeks / 52)
        + np.random.normal(0, 700, 104))

# Create time series
data = pd.Series(users, index=dates)

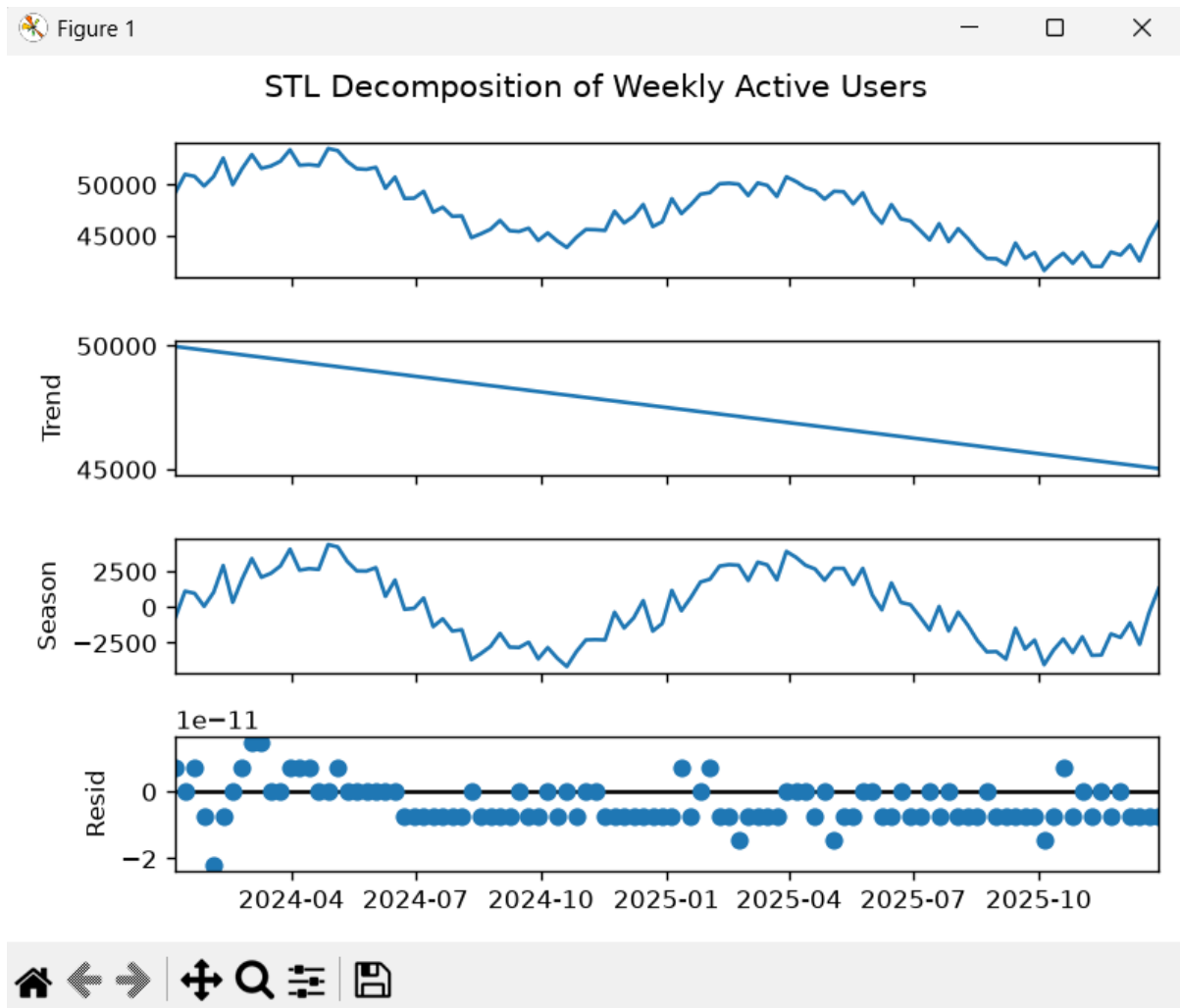
# Apply STL decomposition
stl = STL(data, period=52)
result = stl.fit()

# Plot decomposition
result.plot()
```

```
plt.suptitle("STL Decomposition of Weekly Active Users")
```

```
plt.tight_layout()
```

```
plt.show()
```



9. A stock trading firm wants to reduce short-term noise in a volatile stock's price data to

spot the underlying momentum before making buy/sell calls. Using R, compute and plot a 7-

day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

```
# Create 60 days of sample stock prices
```

```
set.seed(123)
```

```
Date <- seq(as.Date("2025-01-01"), by = "day", length.out = 60)
```

```
Price <- 100 + cumsum(rnorm(60, 0.5, 3))
```

```
# Calculate 7-day moving average
```

```
MA7 <- c(rep(NA, 6),  
        stats::filter(Price, rep(1/7, 7), sides = 2))
```

```
# Calculate 30-day moving average
```

```
MA30 <- c(rep(NA, 29),  
          stats::filter(Price, rep(1/30, 30), sides = 2))
```

```
# Plot raw price
```

```
plot(Date, Price,  
     type = "l",  
     col = "gray",  
     main = "Stock Price with Moving Averages",  
     xlab = "Date",  
     ylab = "Stock Price")
```

```
# Add moving averages
```

```
lines(Date, MA7, col = "blue", lwd = 2)
```

```
lines(Date, MA30, col = "red", lwd = 2)
```

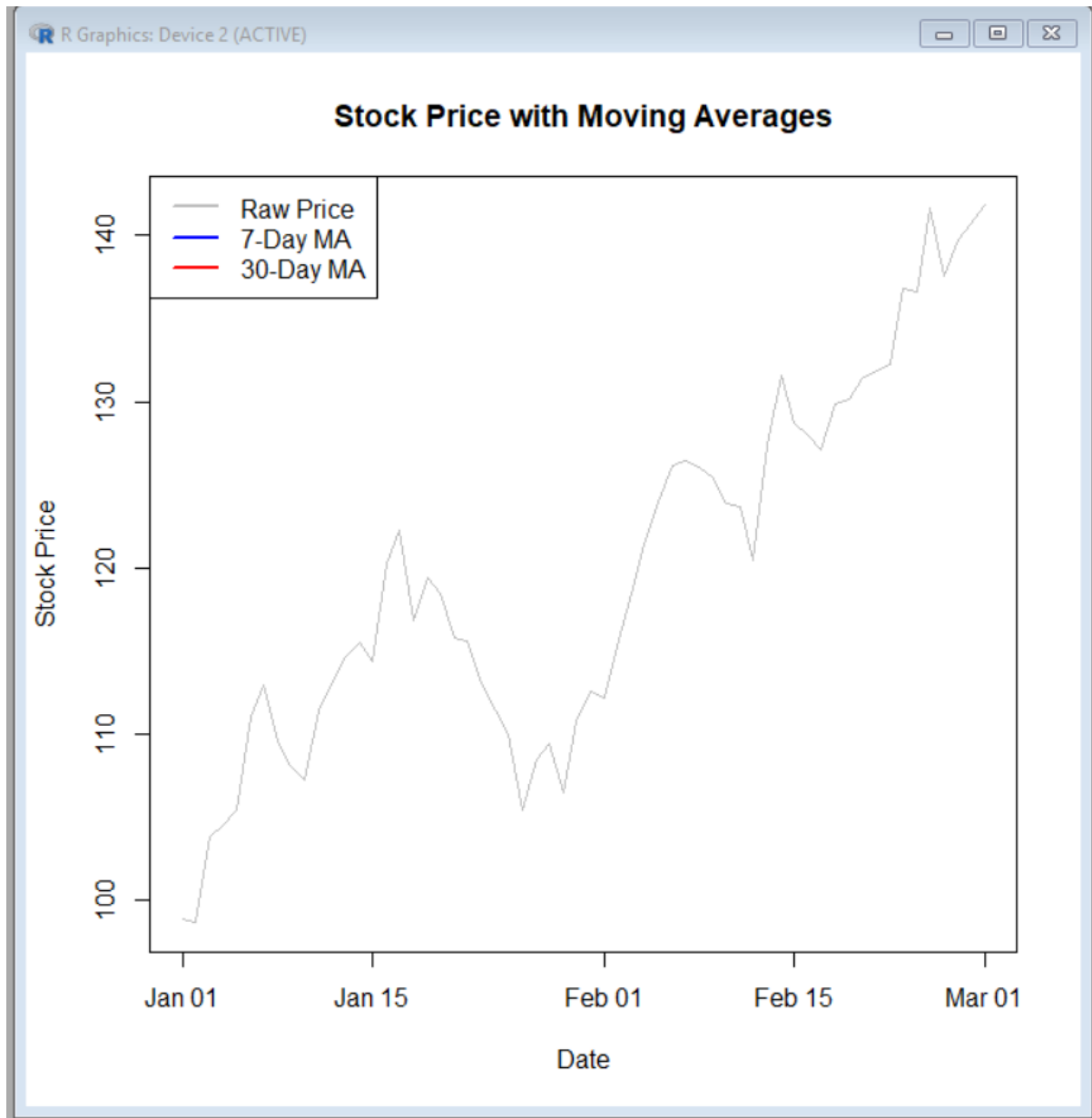
```
# Add legend
```

```
legend("topleft",  
      legend = c("Raw Price", "7-Day MA", "30-Day MA"),
```



```
col = c("gray", "blue", "red"),
```

```
lwd = 2)
```



10. An economist is analyzing a country's quarterly GDP data but needs to remove the long-

term growth trend to study business-cycle fluctuations in isolation. Using Python, detrend the

GDP series (e.g., by differencing or regression-based detrending), plot the detrended series,

and interpret the remaining fluctuations.

```
import numpy as np

import matplotlib.pyplot as plt

# Create 20 quarters
quarters = np.arange(1, 21)

# Sample GDP data
np.random.seed(123)
GDP = 500 + 8 * quarters + 20 * np.sin(quarters) + np.random.normal(0, 8, 20)

# Fit a linear trend
coefficients = np.polyfit(quarters, GDP, 1)
trend = np.polyval(coefficients, quarters)

# Detrend the GDP series
detrended_GDP = GDP - trend

# Plot detrended GDP
plt.plot(quarters, detrended_GDP, marker='o')

# Zero reference line
plt.axhline(0, linestyle='--')

plt.xlabel("Quarter")
plt.ylabel("Detrended GDP")
```

```
plt.title("Detrended Quarterly GDP")
```

```
plt.grid(True)
```

```
plt.show()
```

